

Neural Network

**By,
Dr. Vimal Shrivastava
Assistant Professor
School of Electronics Engg.
KIIT Bhubaneswar**

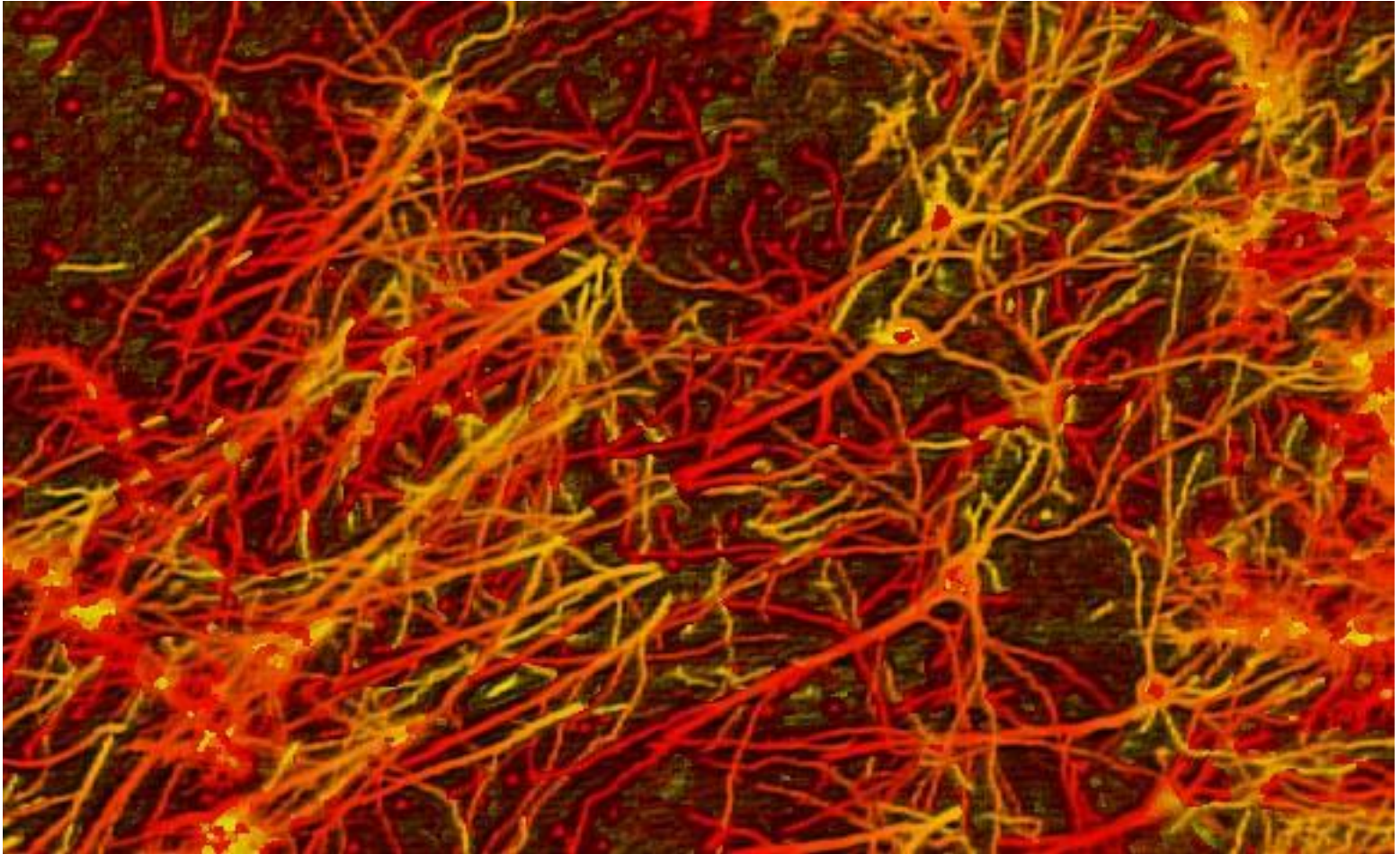
Introduction To Biological Neural Network

- The word “**NN**” drives its origin from **Human Brain** or **Human Nervous System**.
- Human Brain consists of large number of **neurons and their parallel interconnections**.
- That achieves different task such as recognition with unexpected small amount of time.
- This is what inspired to researchers that “**Is there any way to mimic the learning like human brain with massive interconnections**”.

Brain Computation

- **Brain Structure:** A highly complex, non-linear and parallel interconnections.
- The human brain contains about **10 billion nerve cells or neurons**.
- On an average, each neuron is connected to other neurons through approximately **10,000 synapses**.
- Overall, there are **60 trillion** (approx) interconnections.

Interconnections in Brain



Biological Neurons

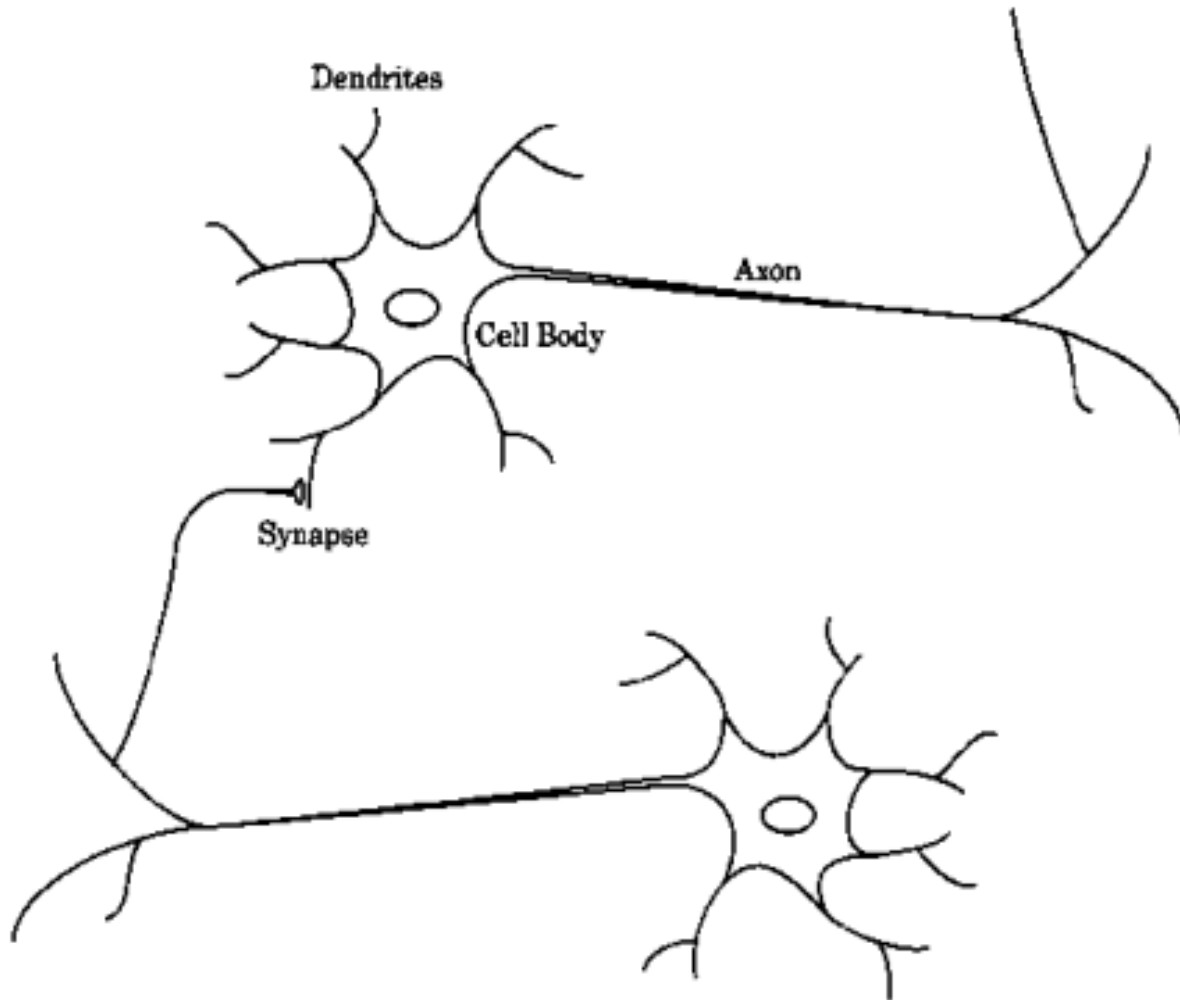


Figure 1.1 Schematic Drawing of Biological Neurons

Biological Neurons (cont..)

- Neurons have four principal components:
 - Dendrites
 - Cell Body
 - Axon
 - Synapses
- The **dendrites** are tree-like networks of nerve fibers that carry electrical signals into the cell body.
- The **cell body** effectively sums and thresholds these incoming signals.
- The **axon** is single long fiber that carries the signal from the cell body out to other neurons.

Biological Neurons (cont..)

- The point of contact between an axon of one cell and a dendrite of another cell is called “**Synapse**”.
- It is the arrangement of neurons and the strength of individual synapses, determined by a complex chemical process, that establishes the function of the neural network.
- Some of the neural structure is defined at birth. Other parts are developed through learning, as new connections are made and others waste away.
- Neural structures continue to change throughout life. These later changes tend to consists mainly of strengthening or weakening of synaptic junctions.

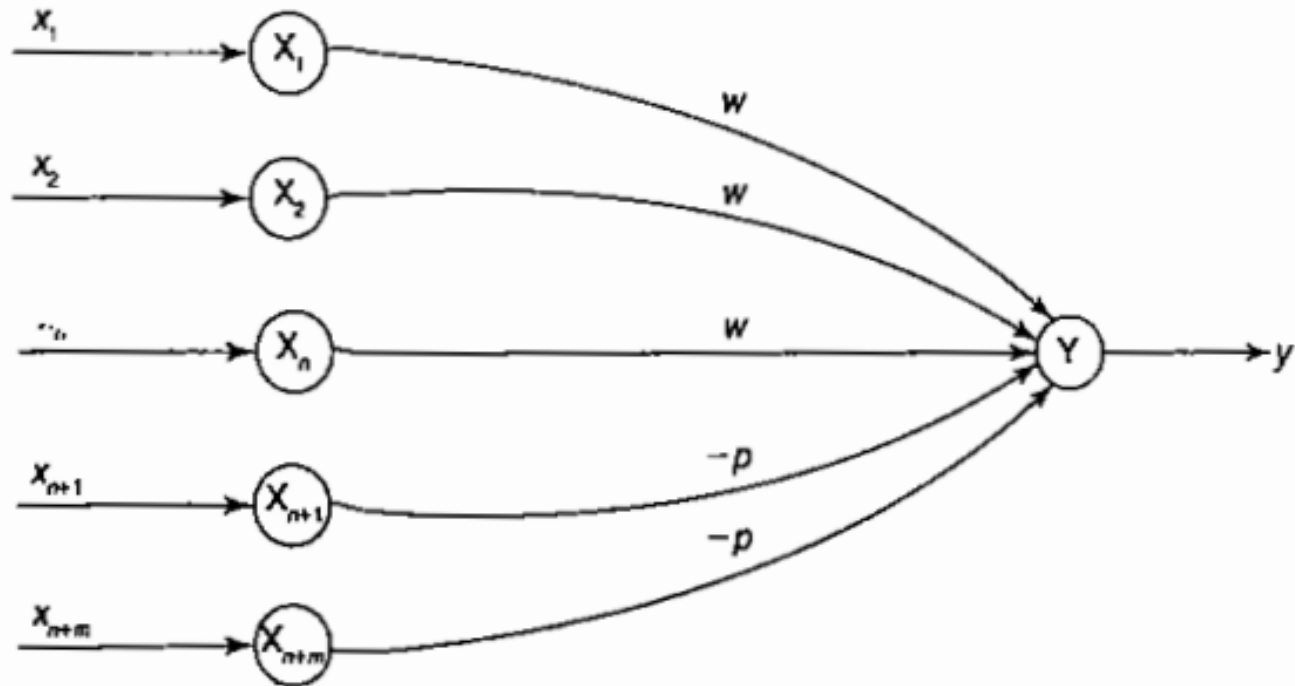
Introduction To Artificial Neural Network (ANN)

ANN

- ANN are gross simplifications of real (biological) networks of neurons.
- The aim of ANN is to mimic the human ability to adapt to changing circumstances and the current environment.
- This depends heavily on being able to learn from events that happened in the past (training) and to be apply this to future situations (testing).
- **Ex:** An experience doctor is far more likely to make a sound decision than a trainee, because of his past experience.
- Similarly, it would be beneficial if machines too could use past events and learn and made a decision on future events. This is the role that ANN seek to fill.

McCulloch-Pitts Neuron Model

- First ANN discovered in 1943.
- Usually called as M-P neuron.
- The activation function of M-P neuron is binary i.e. neuron may fire or may not fire.
- The weights associated with the communication links may be excitatory (positive weight) or inhibitory (negative weight).
- It has no particular training algorithm.
- An analysis has to be performed to determine the values of weights and threshold.
- Mostly used in case of logic functions.



$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$

- **Problem 1:** Implement AND function using McCulloch-Pitts neuron (take binary input).

Solution: Consider the truth table for AND function (Table 1).

Table 1

x_1	x_2	y
1	1	1
1	0	0
0	1	0
0	0	0

Assume, weights be $w_1=1$ and $w_2=1$.

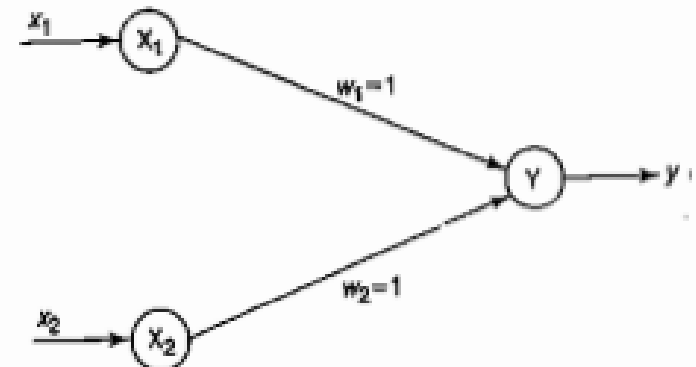
Then, the net input is calculated for four inputs:

$$(1, 1), \quad y_{in} = x_1 w_1 + x_2 w_2 = 1 \times 1 + 1 \times 1 = 2$$

$$(1, 0), \quad y_{in} = x_1 w_1 + x_2 w_2 = 1 \times 1 + 0 \times 1 = 1$$

$$(0, 1), \quad y_{in} = x_1 w_1 + x_2 w_2 = 0 \times 1 + 1 \times 1 = 1$$

$$(0, 0), \quad y_{in} = x_1 w_1 + x_2 w_2 = 0 \times 1 + 0 \times 1 = 0$$



For AND function, the output is high if both the inputs are high.

For this condition, the net input is calculated as 2.

Hence, based on the net input, the threshold is set as 2.

Thus, the output of neuron Y can be written as

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq 2 \\ 0 & \text{if } y_{in} < 2 \end{cases}$$

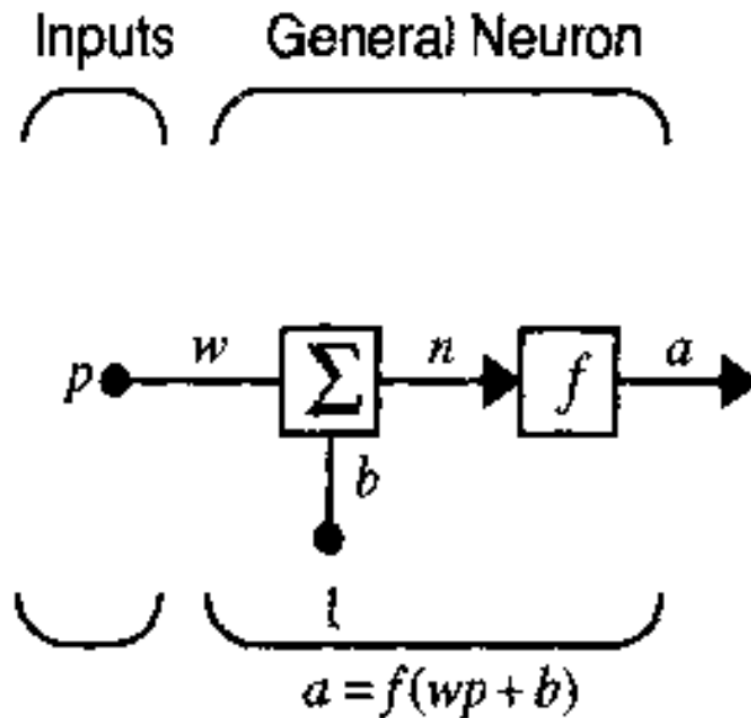
- **Problem 2:** Implement XOR function using McCulloch-Pitts neuron (consider binary data).
- **Problem 3:** Implement ANDNOT function using McCulloch-Pitts neuron (consider binary data).

Solution: In the case of ANDNOT function, the response is true if the first input is true and the second input is false. For all other input variations, the response is false. The truth table for ANDNOT function is given in Table 2.

X1	X2	y
0	0	0
0	1	0
1	0	1
1	1	0

Perceptron Model

Single Input Neuron



The neuron output is calculated as

$$a = f(wp + b) .$$

If, for instance, $w = 3$, $p = 2$ and $b = -1.5$, then

$$a = f(3(2) - 1.5) = f(4.5)$$

Analogy between BNN and ANN

BNN	ANN
Cell	Neuron
Dendrites	Net Input
Synapses	Weights
Axon	Output

Activation Function

- **Activation Function:**
- An Activation Function decides whether a neuron should be activated or not.
- The non-linear activation function is used to ensure that a neuron's output is bounded.

Activation Function / Transfer Function

Name	Input/Output Relation	Icon	MATLAB Function
Hard Limit	$a = 0 \quad n < 0$ $a = 1 \quad n \geq 0$		hardlim
Symmetrical Hard Limit	$a = -1 \quad n < 0$ $a = +1 \quad n \geq 0$		hardlims
Linear	$a = n$		purelin
Saturating Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n \leq 1$ $a = 1 \quad n > 1$		satlin
Symmetric Saturating Linear	$a = -1 \quad n < -1$ $a = n \quad -1 \leq n \leq 1$ $a = 1 \quad n > 1$		satlins

Activation Function / Transfer Function (cont..)

Log-Sigmoid	$a = \frac{1}{1 + e^{-n}}$		logsig
Hyperbolic Tangent Sigmoid	$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$		tansig
Positive Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n$		poslin
Competitive	$a = 1$ neuron with max n $a = 0$ all other neurons		compet

Problems

P2.1 The input to a single-input neuron is 2.0, its weight is 2.3 and its bias is -3.

- i. What is the net input to the transfer function?**
- ii. What is the neuron output?**

P2.2 What is the output of the neuron of P2.1 if it has the following transfer functions?

- i. Hard limit**
- ii. Linear**
- iii. Log-sigmoid**

Solutions

P2.1: i. The net input is given by:

$$n = wp + b = (2.3)(2) + (-3) = 1.6$$

ii. The output cannot be determined because the transfer function is not specified.

P2.2: i. For the hard limit transfer function:

$$a = \text{hardlim}(1.6) = 1.0$$

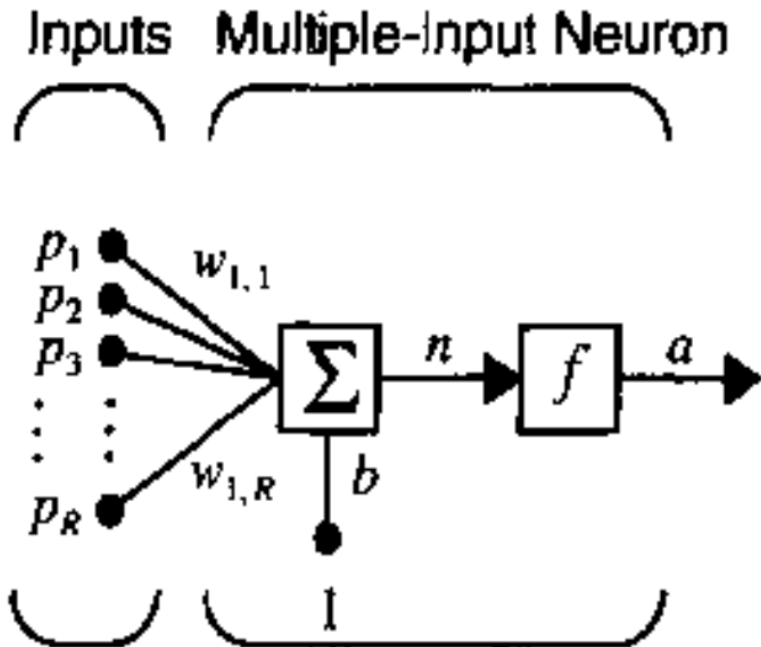
ii. For the linear transfer function:

$$a = \text{purelin}(1.6) = 1.6$$

iii. For the log-sigmoid transfer function:

$$a = \text{logsig}(1.6) = \frac{1}{1 + e^{-1.6}} = 0.8320$$

Multiple Inputs Neuron



$$a = f(\mathbf{W}\mathbf{p} + b)$$

$$n = w_{1,1}p_1 + w_{1,2}p_2 + \dots + w_{1,R}p_R + b.$$

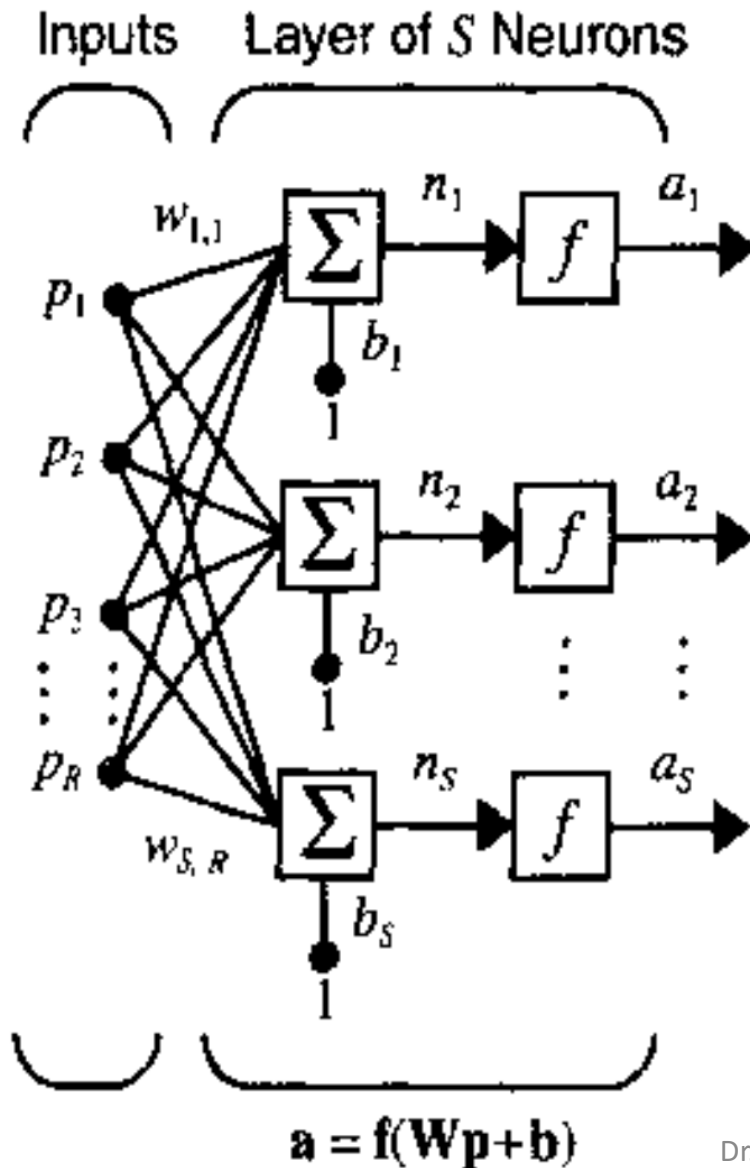
This expression can be written in matrix form:

$$n = \mathbf{W}\mathbf{p} + b,$$

Now the neuron output can be written as

$$a = f(\mathbf{W}\mathbf{p} + b).$$

Multiple Inputs Multiple Neurons



Weight Matrix

$$\mathbf{W} = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}$$

Problems (cont..)

P2.3 Given a two-input neuron with the following parameters: $b = 1.2$, $W = \begin{bmatrix} 3 & 2 \end{bmatrix}$ and $p = \begin{bmatrix} -5 & 6 \end{bmatrix}^T$, calculate the neuron output for the following transfer functions:

- i. A symmetrical hard limit transfer function**
- ii. A saturating linear transfer function**
- iii. A hyperbolic tangent sigmoid (tansig) transfer function**

P2.4 A single-layer neural network is to have six inputs and two outputs. The outputs are to be limited to and continuous over the range 0 to 1. What can you tell about the network architecture? Specifically:

- i. How many neurons are required?**
- ii. What are the dimensions of the weight matrix?**
- iii. What kind of transfer functions could be used?**

Solutions

P2.3: First calculate the net input n :

$$n = \mathbf{W}\mathbf{p} + b = \begin{bmatrix} 3 & 2 \end{bmatrix} \begin{bmatrix} -5 \\ 6 \end{bmatrix} + (1.2) = -1.8.$$

Now find the outputs for each of the transfer functions.

i. $a = \text{hardlims}(-1.8) = -1$

ii. $a = \text{satlin}(-1.8) = 0$

iii. $a = \text{tansig}(-1.8) = -0.9468$

P2.4: **i.** Two neurons, one for each output, are required.

ii. The weight matrix has two rows corresponding to the two neurons and six columns corresponding to the six inputs. (The product $\mathbf{W}\mathbf{p}$ is a two-element vector.)

iii. Of the transfer functions we have discussed, the *logsig* transfer function would be most appropriate.

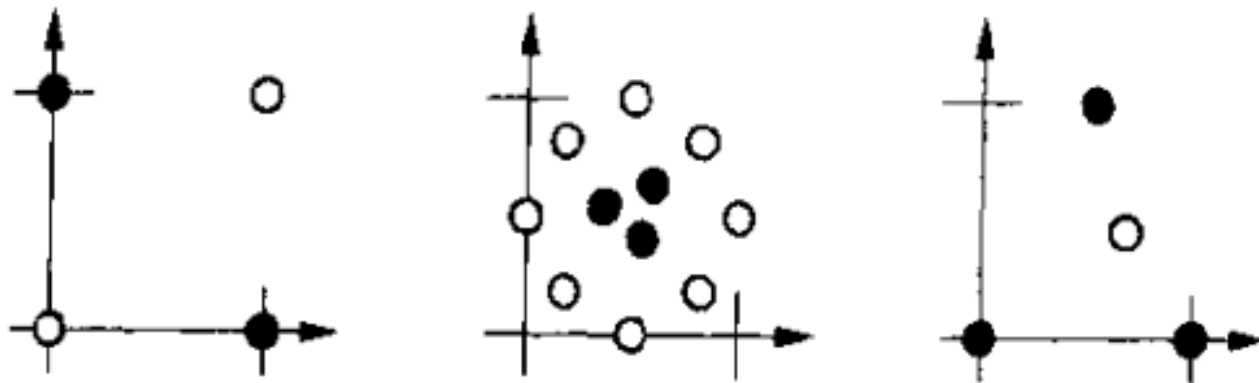
Limitations of Perceptron

- Unable to solve non-linearly separable problem.

Unfortunately, many problems are not linearly separable. The classic example is the XOR gate. The input/target pairs for the XOR gate are

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, t_1 = 0 \right\} \left\{ \mathbf{p}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, t_2 = 1 \right\} \left\{ \mathbf{p}_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, t_3 = 1 \right\} \left\{ \mathbf{p}_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, t_4 = 0 \right\}.$$

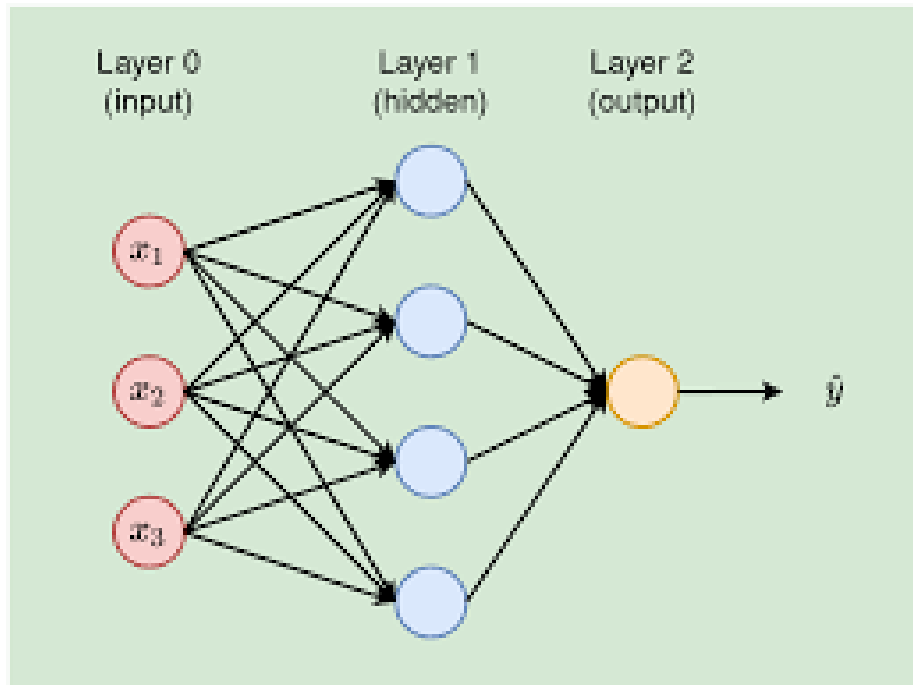
This problem is illustrated graphically on the left side of Figure 4.6, which also shows two other linearly inseparable problems. Try drawing a straight line between the vectors with targets of 1 and those with targets of 0 in any of the diagrams of Figure 4.6.



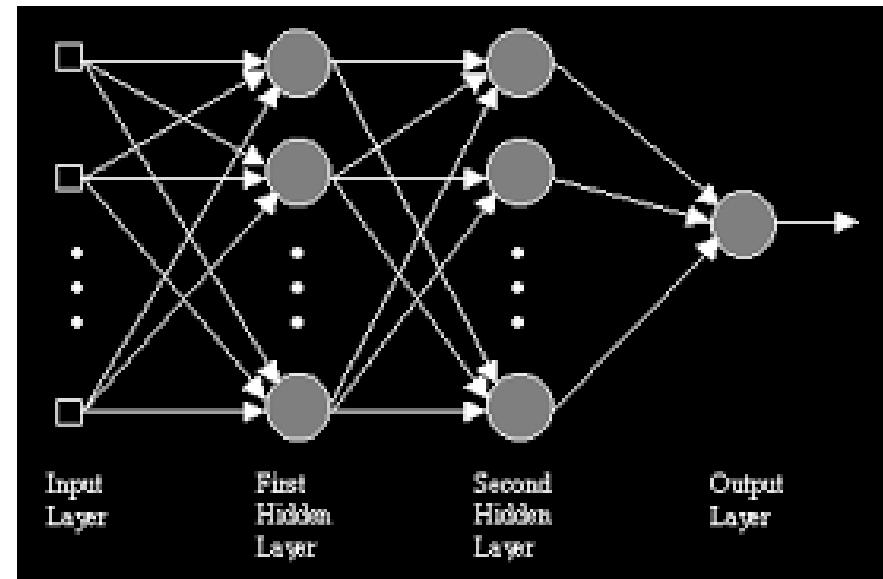
Dr. Vimal K. Shrivastava
Figure 4.6 Linearly Inseparable Problems

Multi-Layer Perceptron (MLP)

- Two Layer Network

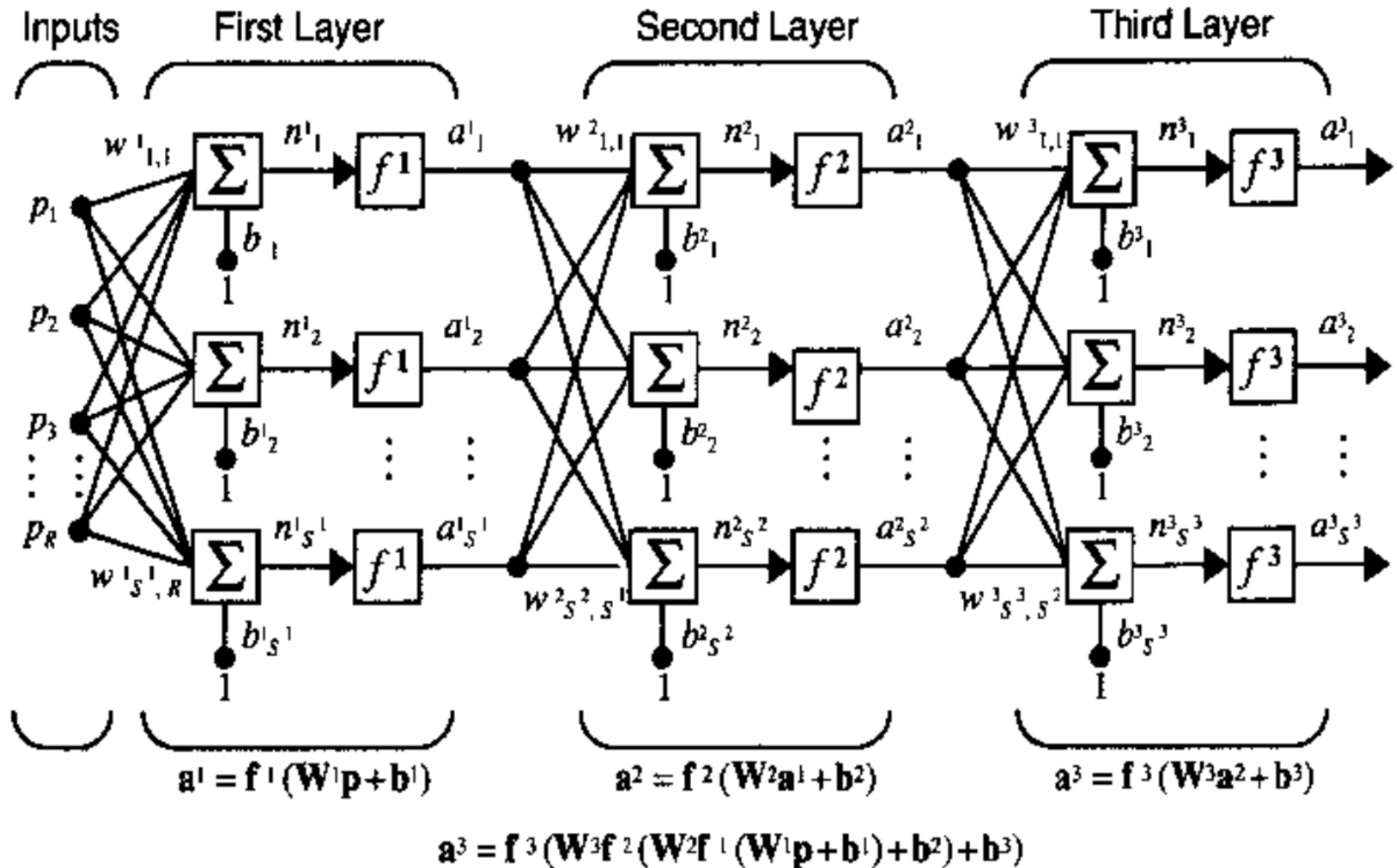


- Three Layer Network

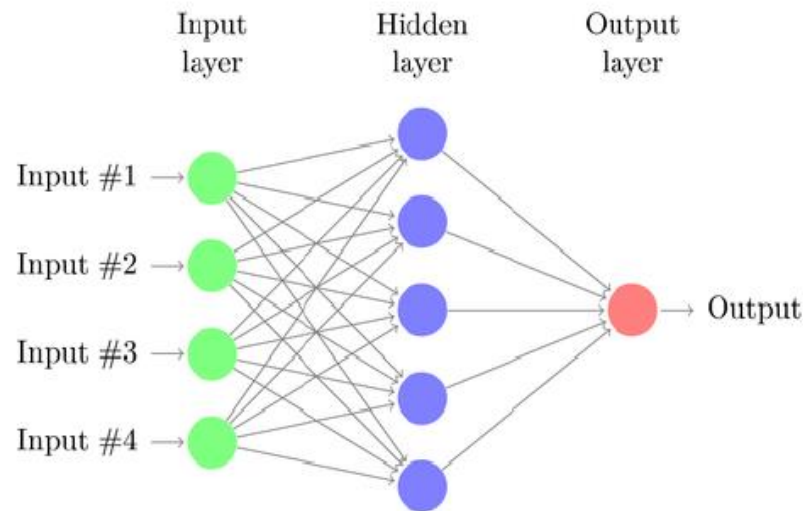


Multi-Layer Perceptron (MLP)

- Three Layer Network:



Problem:



- Find out the number of parameters to be trained.

Solution:

Number of weights between input and hidden layer:

*Number of inputs * Number of Neurons in the hidden layer = $4 \times 5 = 20$*

Number of weights between hidden layer and output layer:

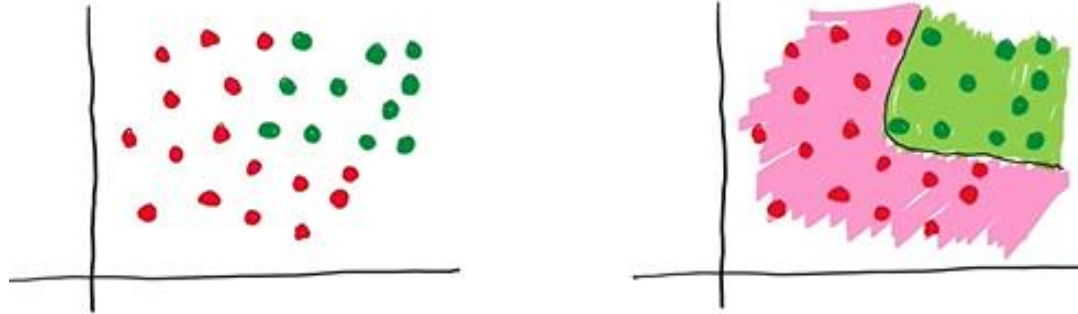
*Number of Neurons in the hidden layer * Number of Neurons in o/p Layer = $5 \times 1 = 5$*

Number of biases for hidden layer: *Number of Neurons in hidden layer = 5*

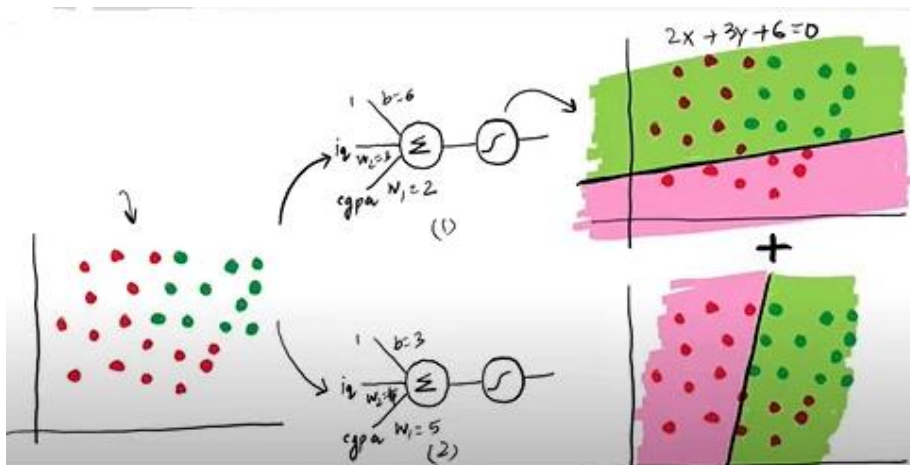
Number of biases for output layer: *Number of Neurons in Output Layer = 1*

Hence, Total number of parameters to be trained = $20 + 5 + 5 + 1 = 31$

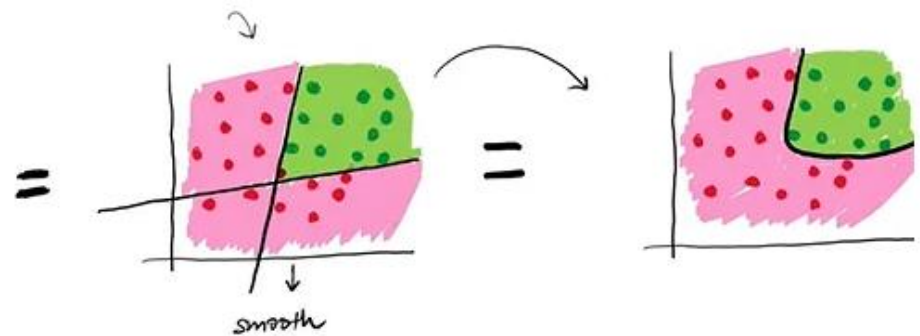
How MLP Captures Non-Linearity in Data



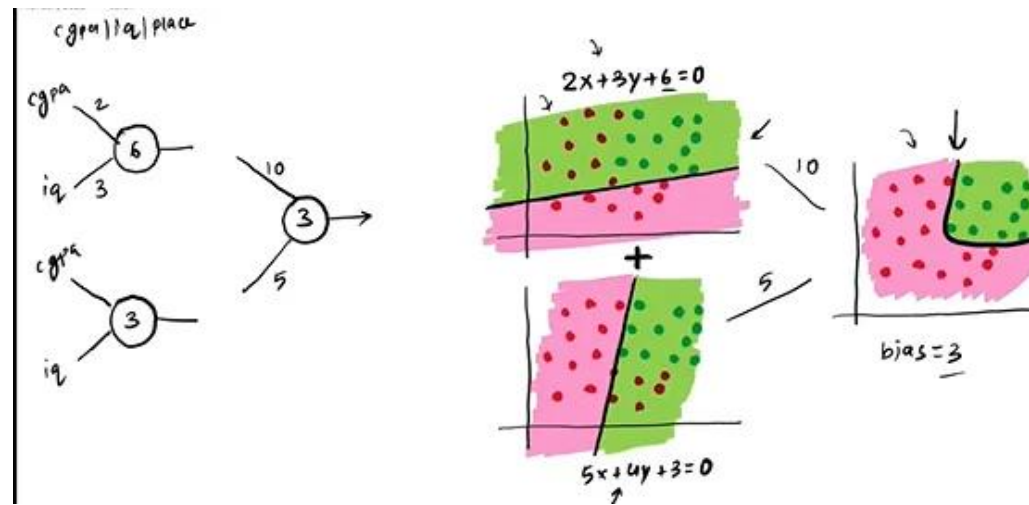
Dataset with non-linear decision boundary



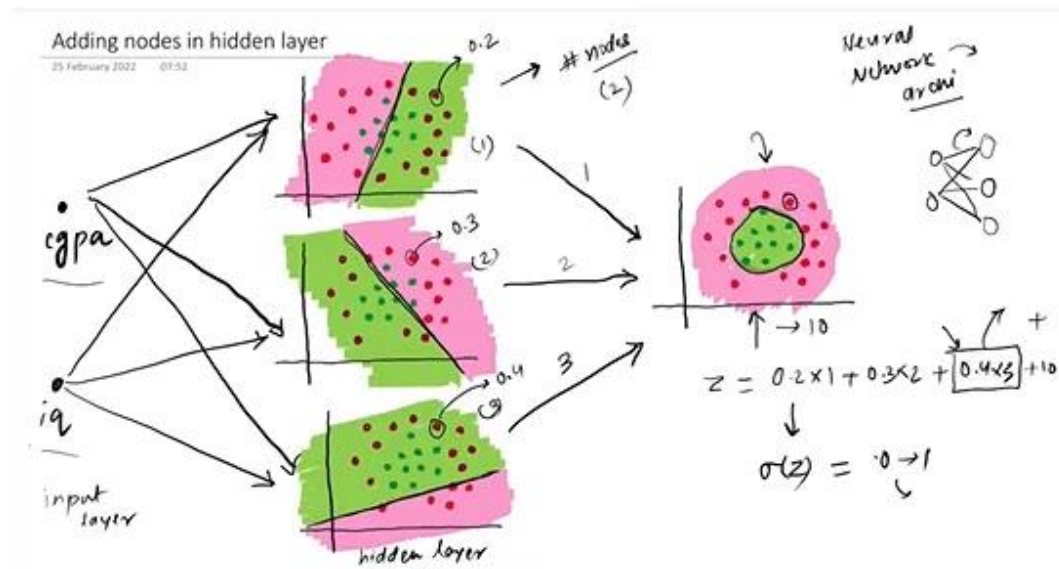
Two different single layer perceptron with their decision boundary on non linear datasets.



Superimposing and smoothing the linear decision boundary of single layer perceptron to form non linear decision boundary as MLP



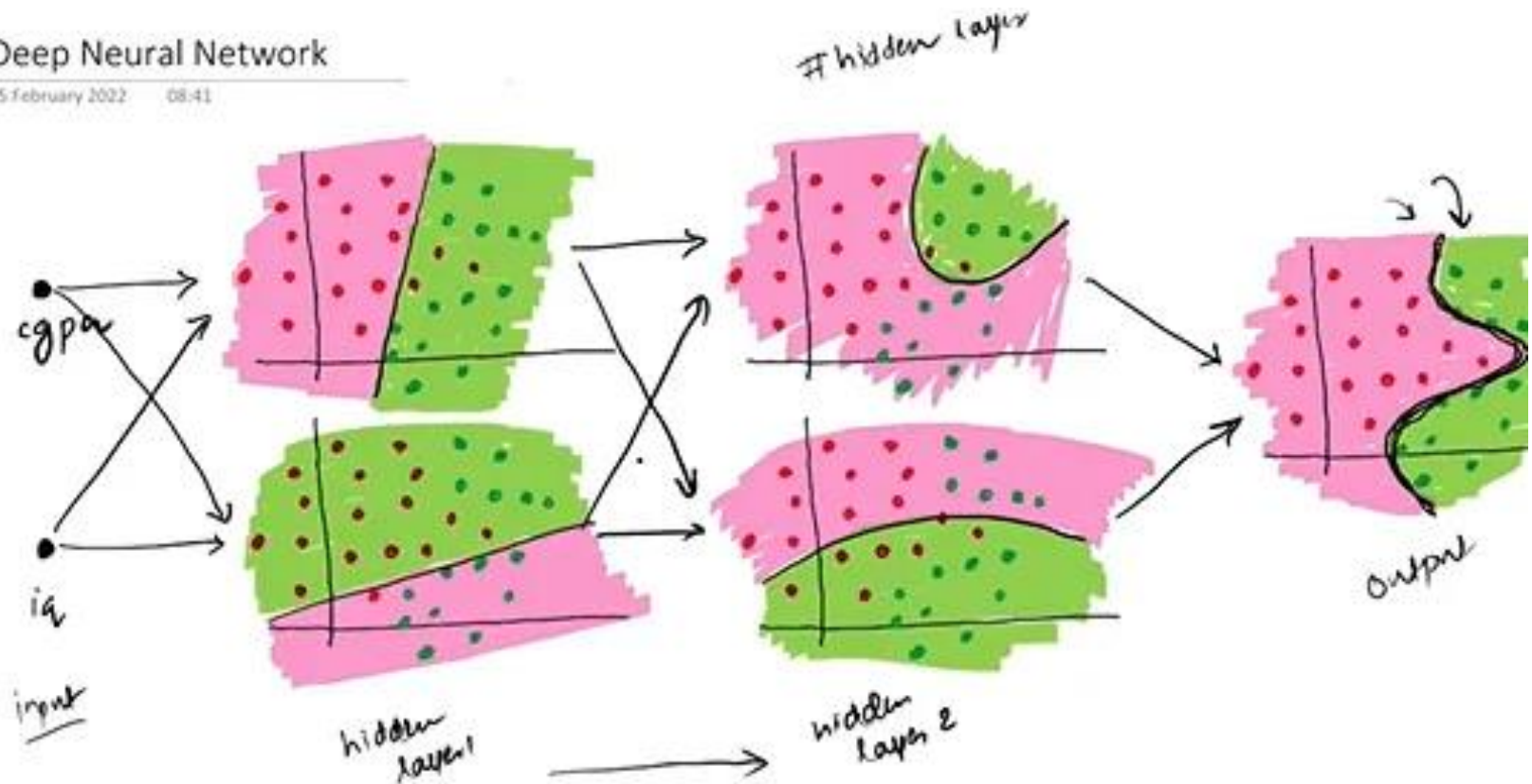
A big picture of MLP and how on superimposition it makes non linear decision boundary.



Addiong nodes to hidden layer ables to capture more complex pattern.

Deep Neural Network

25 February 2022 08:41



Deep Neural Networks with multiple hidden layers to solve even more complex decision boundary.

<https://medium.com/@amanatulla1606/unraveling-the-magic-how-multilayer-perceptron-captures-non-linearity-in-data-6a4d385f7592>

Back-Propagation Algorithm

- The back-propagation learning algorithm is one of the most important developments in ANN.
- The basic concept of back-propagation algorithm is **gradient descent** method.
- The error between predicted output and desired output is calculated in terms of loss.
- Then, **gradients of the loss function** is computed with respect to weights and back-propagated to update the parameters.

$$\left[\frac{\partial(Loss)}{\partial W}, \frac{\partial(Loss)}{\partial b} \right]$$

$$W_{new} = W_{old} - \alpha \frac{\partial(Loss)}{\partial W}$$

$$b_{new} = b_{old} - \alpha \frac{\partial(Loss)}{\partial b}$$

where, α is learning rate.

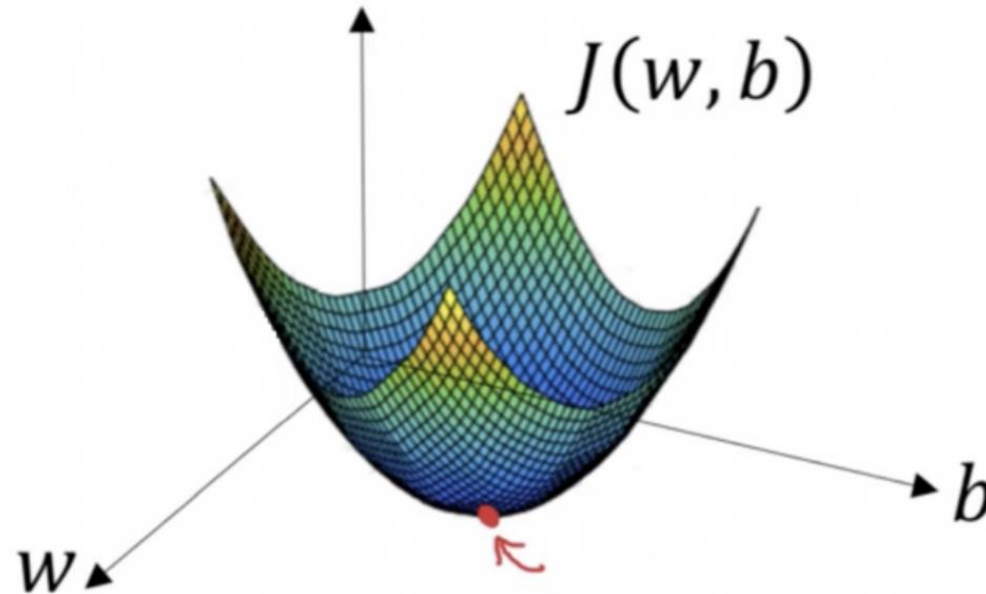
Back-Propagation Algorithm (cont..)

- Initialize all the weights with small random values.
- Feed data into the network and figure out the value of the loss (error) function, obtained by comparison with the expected output value.
- The **gradients of the loss function** with respect to each weight is calculated.
- Each weight and bias is updated in an iterative way.
- We keep recalculating the gradients at the beginning of each training iteration step, until the error becomes lower than a certain established threshold, or the maximum number of epochs is reached.

Gradient Descent

- Gradient descent is an optimization algorithm that is used for training a model.
- It is used to find the values of a function's parameters that minimize a cost function as far as possible.
- It is used for finding a local minimum of a differentiable function.
- **WHAT IS A GRADIENT?**
- A gradient measures how much the output of a function changes if you change the inputs a little bit.
- It measures the change in all weights with regard to the change in error (loss).
- It acts as slope of a function. The higher the gradient, the steeper the slope and the faster a model can learn.

- In mathematical terms, a gradient is a partial derivative with respect to its inputs.



- Let's say, we want to solve a machine learning problem and train our algorithm with gradient descent to minimize the cost function $J(w, b)$ and reach its local minimum by tweaking its parameters (w and b).
- The image above shows the horizontal axes represent the parameters (w and b), while the cost function $J(w, b)$ is represented on the vertical axes.

- We want to find the values of w and b that correspond to the minimum of the cost function (marked with the red arrow).
- We initialize w and b with some random numbers.
- Gradient descent then starts at that point and it takes one step after another in the steepest downside direction until it reaches the point where the cost function is as small as possible.
- Then, gradients of the loss function is computed with respect to weights and back-propagate to update the parameters.

$$\left[\frac{\partial(Loss)}{\partial W}, \frac{\partial(Loss)}{\partial b} \right]$$

$$W_{new} = W_{old} - \alpha \frac{\partial(Loss)}{\partial W}$$

$$b_{new} = b_{old} - \alpha \frac{\partial(Loss)}{\partial b}$$

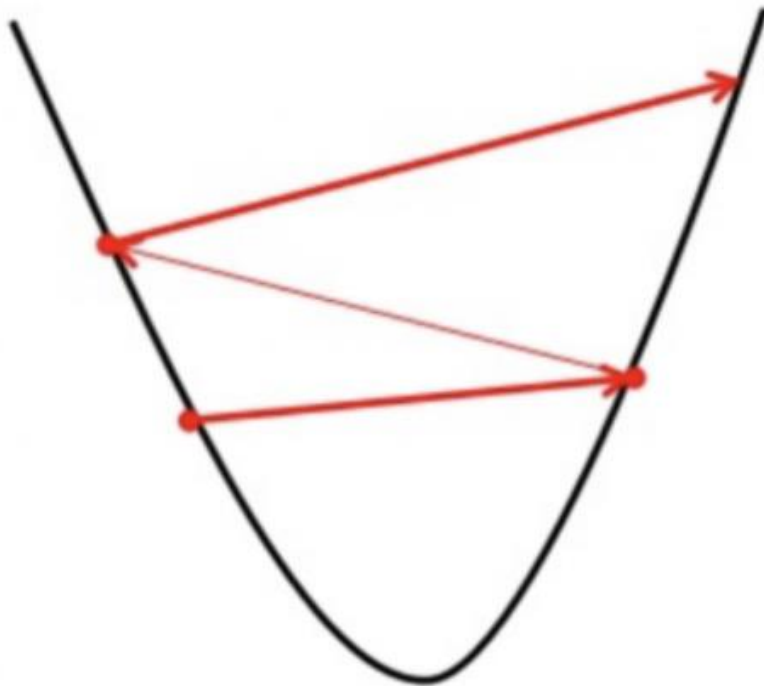
where, α is learning rate.

Importance of Learning Rate

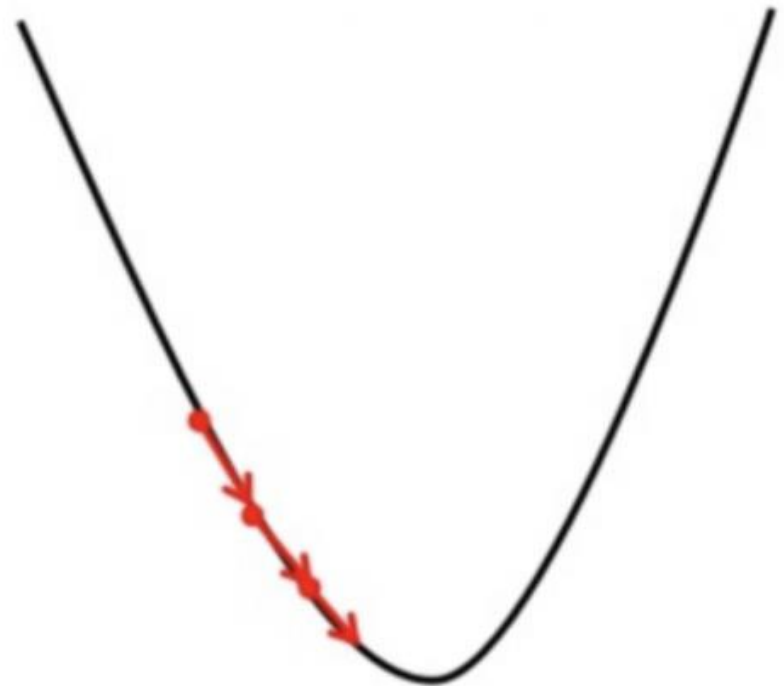
- In gradient descent, the steps into the direction of local minimum are determined by the learning rate, which decides how fast or slow we will move towards the optimal weights.
- If we set the learning rate to too high, the steps it takes will be too big, it may not reach the local minimum because it bounces back and forth between the function of gradient descent.
- If we set the learning rate to a very small value, gradient descent will eventually reach the local minimum but that may take a long time (computationally complex).
- Therefore, we must set the learning rate to an appropriate value, which is neither too low nor too high.

Importance of Learning Rate (cont..)

Big learning rate



Small learning rate

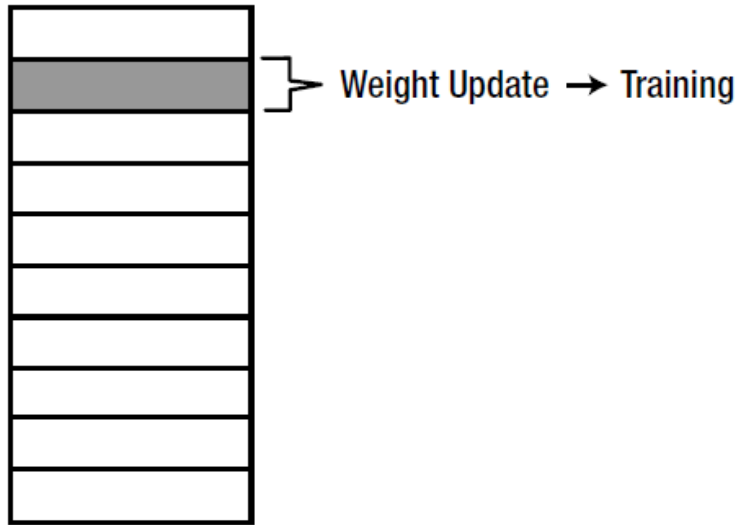


Types of Gradient Descent

- There are three popular types of gradient descent technique that mainly differ in the amount of data they use.
- **Stochastic Gradient Descent (SGD):**
 - It calculates the error for each training data and adjusts the weights immediately.
 - If we have 100 training data points, the SGD adjusts the weights 100 times.
- **Batch Gradient Descent:**
 - It calculates the error for each data point within the training dataset, but update the parameters only after all training examples have been evaluated.
 - This method uses all of the training data and updates only once.

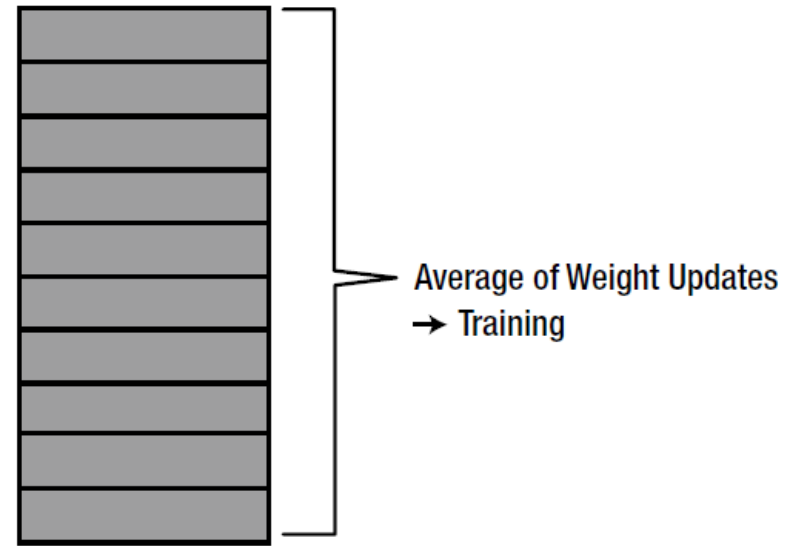
- **Mini-Batch Gradient Descent:**

- The mini batch method is a blend of the SGD and batch methods.
- It simply splits the training dataset into small batches and performs an update for each of those batches.
- Therefore, it calculates the weight updates of the selected data and trains the neural network with the averaged weight update.
- For example, if mini-batch size is 20 and total data points are 100. In this case, a total of five weight adjustments are performed to complete the training process for all the data points ($100/20=5$).



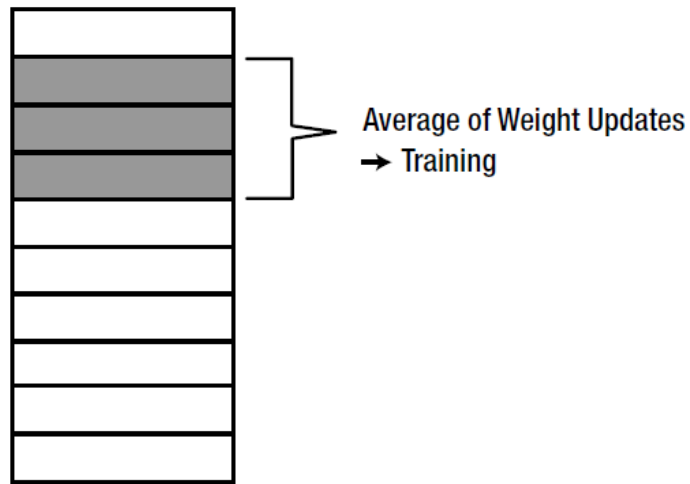
Training Data

Stochastic Gradient Descent



Training Data

Batch Gradient Descent

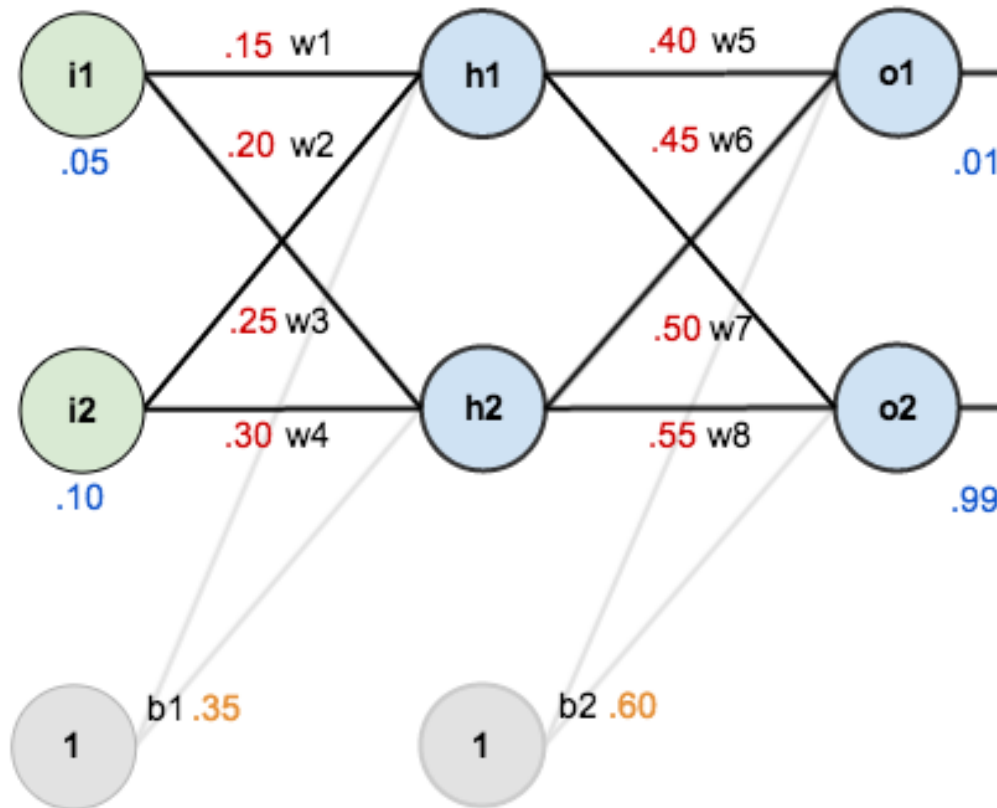


Training Data

Mini-Batch Gradient Descent

Problem on MLP

Using back-propagation network, find the new weights and bias after one epoch for the network shown in below Figure. Use a learning rate $\eta=0.5$ and sigmoid activation function.



<https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>

Solution

1) Forward pass

Hidden pre-activations

$$net_{h1} = 0.05(0.15) + 0.10(0.25) + 0.35 = 0.3825$$

$$net_{h2} = 0.05(0.20) + 0.10(0.30) + 0.35 = 0.3900$$

Hidden activations

$$h_1 = \sigma(0.3825) = 0.5944759307, \quad h_2 = \sigma(0.3900) = 0.5962826993$$

Output pre-activations

$$net_{o1} = h_1(0.40) + h_2(0.45) + 0.60 = 1.1061175870$$

$$net_{o2} = h_1(0.50) + h_2(0.55) + 0.60 = 1.2251934500$$

Outputs

$$o_1 = \sigma(1.106117587) = 0.7514046014, \quad o_2 = \sigma(1.22519345) = 0.7729762086$$

Backward Path:

2) Output-layer deltas and gradients $\delta_{o_k} = (o_k - t_k) o_k(1 - o_k)$

$$\delta_{o1} = (o_1 - t_1)o_1(1 - o_1) = (0.7514046014 - 0.01) \cdot 0.7514046014 \cdot 0.2485953986 = 0.1384912111$$

$$\delta_{o2} = (o_2 - t_2)o_2(1 - o_2) = (0.7729762086 - 0.99) \cdot 0.7729762086 \cdot 0.2270237914 = -0.0380842007$$

Gradients:

$$\frac{\partial E}{\partial w_5} = \delta_{o1}h_1 = 0.1384912111 \cdot 0.5944759307 = 0.0823296916$$

$$\frac{\partial E}{\partial w_6} = \delta_{o1}h_2 = 0.1384912111 \cdot 0.5962826993 = 0.0825799132$$

$$\frac{\partial E}{\partial w_7} = \delta_{o2}h_1 = -0.0380842007 \cdot 0.5944759307 = -0.0226401407$$

$$\frac{\partial E}{\partial w_8} = \delta_{o2}h_2 = -0.0380842007 \cdot 0.5962826993 = -0.0227089500$$

$$\frac{\partial E}{\partial b_2} = \delta_{o1} + \delta_{o2} = 0.1004070103$$

Parameter updates ($\eta = 0.5$)

$$\theta \leftarrow \theta - \eta \frac{\partial E}{\partial \theta}$$

Output layer

$$w'_5 = 0.40 - 0.5(0.0823296916) = \mathbf{0.3588351542}$$

$$w'_6 = 0.45 - 0.5(0.0825799132) = \mathbf{0.4087100434}$$

$$w'_7 = 0.50 - 0.5(-0.0226401407) = \mathbf{0.5113200703}$$

$$w'_8 = 0.55 - 0.5(-0.0227089500) = \mathbf{0.5613544750}$$

$$b'_2 = 0.60 - 0.5(0.1004070103) = \mathbf{0.5497964948}$$

3) Hidden-layer deltas and gradients

$$\delta_{h1} = (\delta_{o1}w_5 + \delta_{o2}w_7) h_1(1 - h_1) = (0.1384912111 \cdot 0.40 + (-0.0380842007) \cdot 0.50) \cdot (0.5944759307)(0.4055240693) = 0.0087641076$$

$$\delta_{h2} = (\delta_{o1}w_6 + \delta_{o2}w_8) h_2(1 - h_2) = (0.1384912111 \cdot 0.45 + (-0.0380842007) \cdot 0.55) \cdot (0.5962826993)(0.4037173007) = 0.0099601250$$

Gradients:

$$\frac{\partial E}{\partial w_1} = \delta_{h1}i_1 = 0.0087641076 \cdot 0.05 = 0.0004382054$$

$$\frac{\partial E}{\partial w_3} = \delta_{h1}i_2 = 0.0087641076 \cdot 0.10 = 0.0008764108$$

$$\frac{\partial E}{\partial w_2} = \delta_{h2}i_1 = 0.0099601250 \cdot 0.05 = 0.0004980063$$

$$\frac{\partial E}{\partial w_4} = \delta_{h2}i_2 = 0.0099601250 \cdot 0.10 = 0.0009960125$$

$$\frac{\partial E}{\partial b_1} = \delta_{h1} + \delta_{h2} = 0.0187242327$$

Parameter updates ($\eta = 0.5$)

Hidden layer

$$w'_1 = 0.15 - 0.5(0.0004382054) = \mathbf{0.1497808973}$$

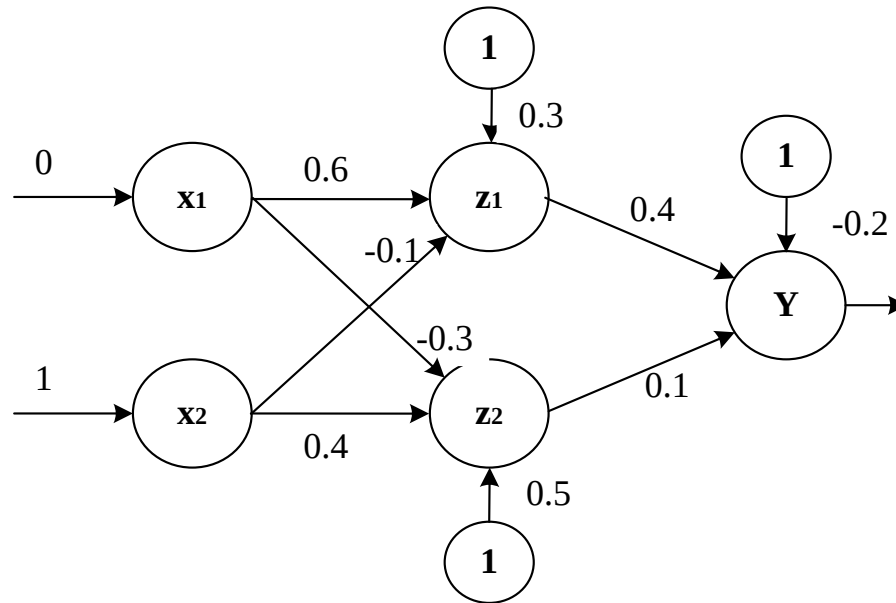
$$w'_2 = 0.20 - 0.5(0.0004980063) = \mathbf{0.1997509969}$$

$$w'_3 = 0.25 - 0.5(0.0008764108) = \mathbf{0.2495617946}$$

$$w'_4 = 0.30 - 0.5(0.0009960125) = \mathbf{0.2995019937}$$

$$b'_1 = 0.35 - 0.5(0.0187242327) = \mathbf{0.3406378837}$$

Problem 2: Using back-propagation network, find the new weights for the net shown in below Figure. It is presented with the input pattern [0, 1] and the target output is 1. Use a learning rate $\eta=0.25$ and binary sigmoidal activation function.



Solution:

initial weights are $[v_{11} \ v_{21} \ v_{01}] = [0.6 \ -0.1 \ 0.3]$,

$[v_{12} \ v_{22} \ v_{02}] = [-0.3 \ 0.4 \ 0.5]$

$[w_1 \ w_2 \ w_0] = [0.4 \ 0.1 \ -0.2]$

$[x_1, x_2] = [0, 1]$

$t = 1$

$f(x) = \frac{1}{1 + e^{-x}}$

Updated weights and bias after one epoch:

- Compute the final weights of the network:

$$v_{11}(\text{new}) = v_{11}(\text{old}) + \Delta v_{11} = 0.6 + 0 = 0.6$$

$$v_{12}(\text{new}) = v_{12}(\text{old}) + \Delta v_{12} = -0.3 + 0 = -0.3$$

$$\begin{aligned} v_{21}(\text{new}) &= v_{21}(\text{old}) + \Delta v_{21} \\ &= -0.1 + 0.00295 = -0.09705 \end{aligned}$$

$$\begin{aligned} v_{22}(\text{new}) &= v_{22}(\text{old}) + \Delta v_{22} \\ &= 0.4 + 0.0006125 = 0.4006125 \end{aligned}$$

$$\begin{aligned} w_1(\text{new}) &= w_1(\text{old}) + \Delta w_1 = 0.4 + 0.0164 \\ &= 0.4164 \end{aligned}$$

$$\begin{aligned} w_2(\text{new}) &= w_2(\text{old}) + \Delta w_2 = 0.1 + 0.02117 \\ &= 0.12117 \end{aligned}$$

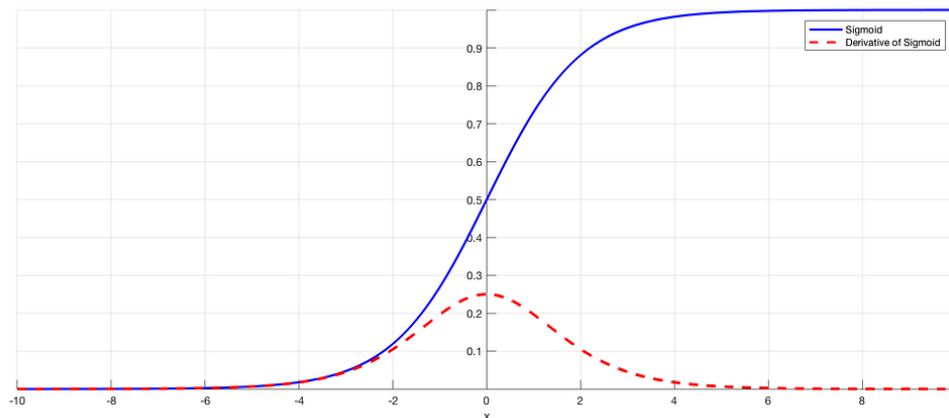
$$\begin{aligned} v_{01}(\text{new}) &= v_{01}(\text{old}) + \Delta v_{01} = 0.3 + 0.00295 \\ &= 0.30295 \end{aligned}$$

$$\begin{aligned} v_{02}(\text{new}) &= v_{02}(\text{old}) + \Delta v_{02} \\ &= 0.5 + 0.0006125 = 0.5006125 \end{aligned}$$

$$\begin{aligned} w_0(\text{new}) &= w_0(\text{old}) + \Delta w_0 = -0.2 + 0.02978 \\ &= -0.17022 \end{aligned}$$

The Vanishing Gradient Problem

- **The problem:**
- As more layers using certain activation functions are added to neural networks, the gradients of the loss function approaches zero, making the network hard to train.
- **Why:**
- Certain activation functions, like the sigmoid function, makes a large input space into a small input space between 0 and 1. Therefore, a large change in the input of the sigmoid function will cause a small change in the output. Hence, the derivative becomes small.

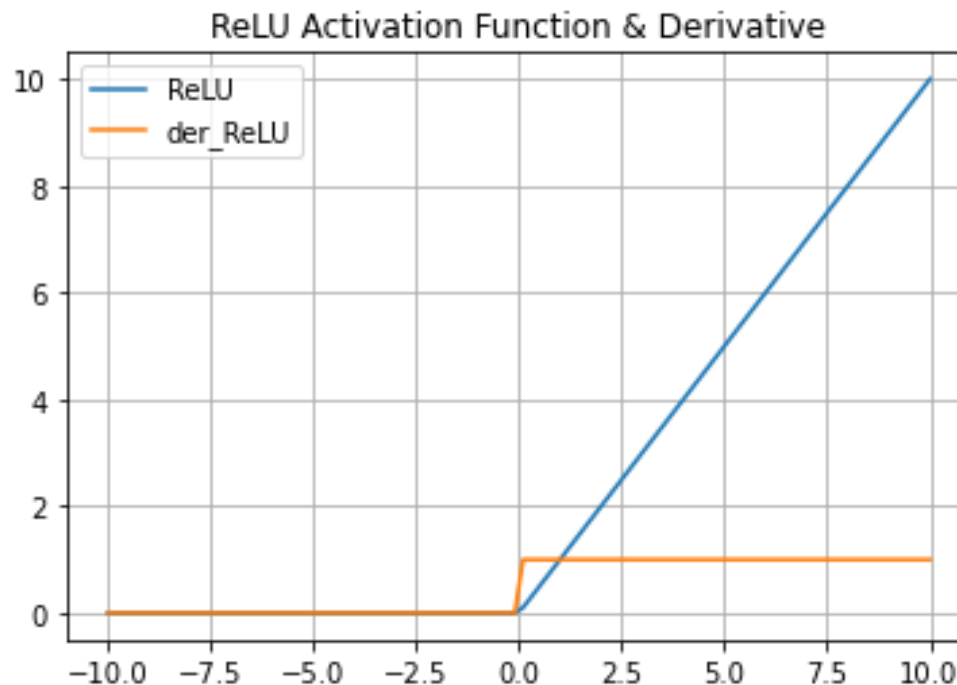


The sigmoid function and its derivative

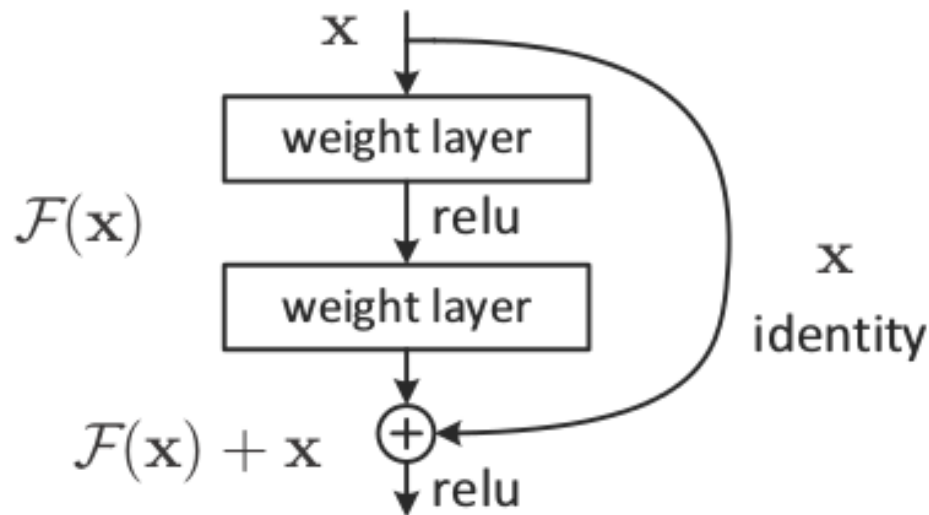
- **Why it's significant:**
- For shallow network with only a few layers that use these activations, this isn't a big problem. However, when more layers are used, it can cause the gradient to be too small for training to work effectively.
- However, when n hidden layers use an activation like the sigmoid function, n small derivatives are multiplied together. Thus, the gradient decreases exponentially as we propagate down to the initial layers.
- A small gradient means that the weights and biases of the initial layers will not be updated effectively with each training session. Since these initial layers are often crucial to recognizing the core elements of the input data, it can lead to overall inaccuracy of the whole network.

- **Solutions:**
- The simplest solution is to use other activation functions, such as Rectified Linear Unit (ReLU), which doesn't cause a small derivative.

$$f(x) = \begin{cases} 1, & x > 0 \\ 0, & \textit{otherwise} \end{cases}$$

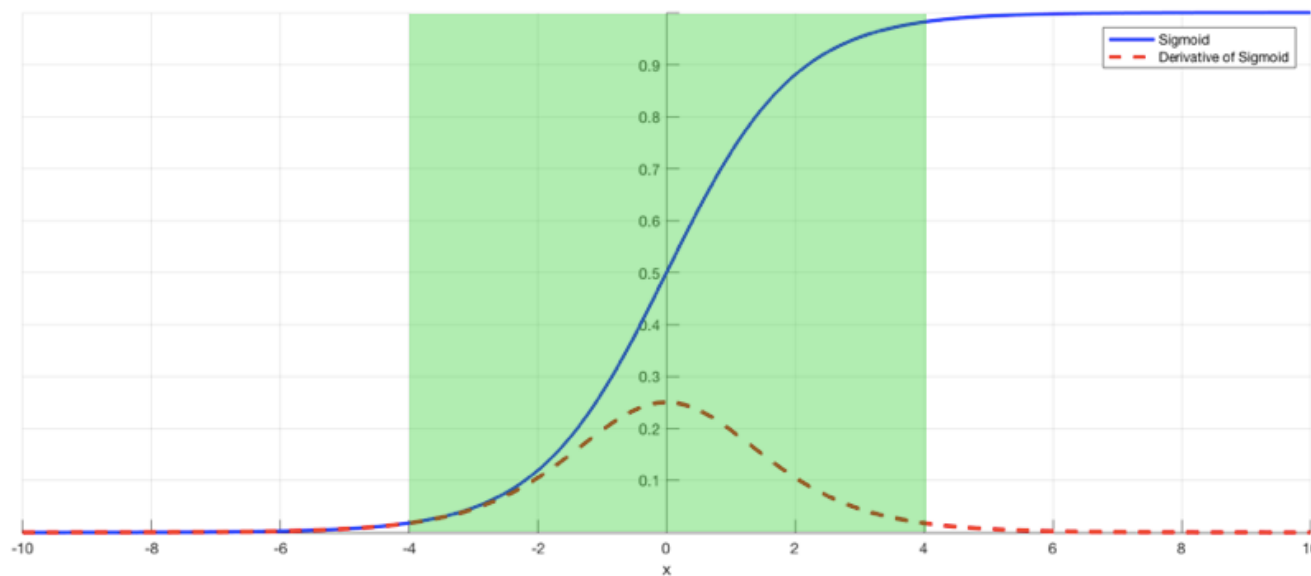


- **Residual network** is another solution, as it provides residual connections straight to earlier layers.
- As seen in figure below, the residual connection directly adds the value at the beginning of the block, $\mathbf{x}$, to the end of the block ($F(\mathbf{x}) + \mathbf{x}$).
- This residual connection doesn't go through activation functions that “squashes” the derivatives, resulting in a higher overall derivative of the block.



A residual block

- Finally, **batch normalization** layers can also resolve the issue.
- As stated before, the problem arises when a large input space is mapped to a small one, causing the derivatives to disappear.
- Batch normalization reduces this problem by simply normalizing the input so $|x|$ doesn't reach the outer edges of the sigmoid function.
- As seen in figure below, it normalizes the input so that most of it falls in the green region, where the derivative isn't too small.



The Exploding Gradient Problem

- If the gradients are large, the multiplication of these gradients will become huge over time. This results in the model being unable to learn and its behavior becomes unstable. This problem is called the exploding gradient problem.
- How to know if model is suffering from the Exploding/Vanishing Gradient Problem?

Exploding

There is an exponential growth in the model parameters.

The model weights may become NaN during training.

The model experiences avalanche learning.

Vanishing

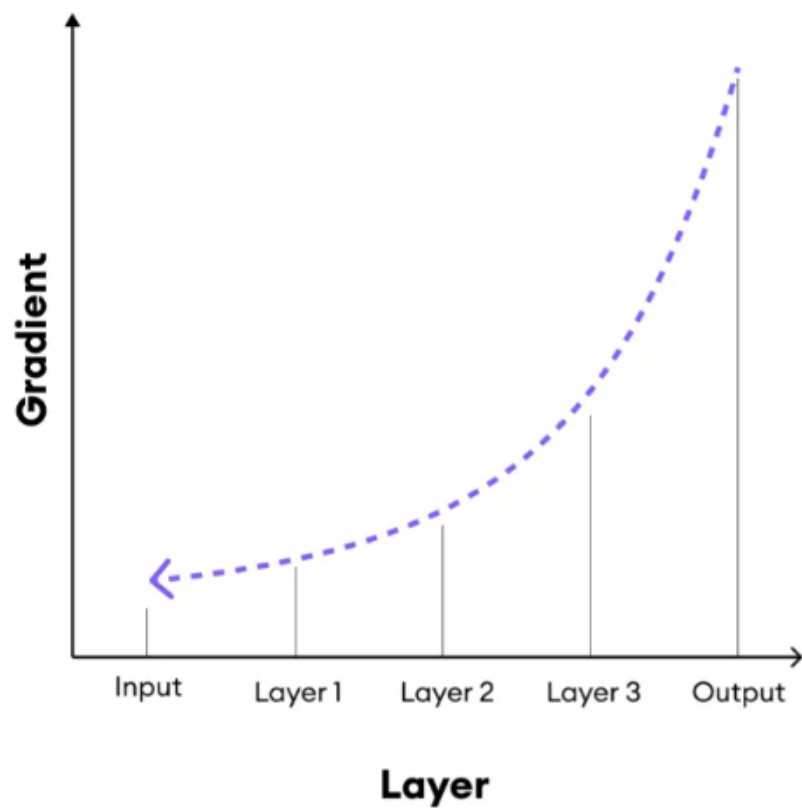
The parameters of the higher layers change significantly whereas the parameters of lower layers would not change much (or not at all).

The model weights may become 0 during training.

The model learns very slowly and perhaps the training stagnates at a very early stage just after a few iterations.

- **Solutions:**
- **Use less number of layers**
- **Use batch normalization**
- **Carefully initialize weights:**
- Choosing the appropriate strategy for initializing the weights can help prevent gradients from exploding at the start of training. Ex: Xavier/Glorot Initialization, He Uniform Initialization.
- **Use gradient clipping:**
- This technique involves clipping the gradients during backpropagation to ensure that they do not exceed a specified threshold
- **Regularization:** Adding a regularization term to the loss function can help to prevent the weights from becoming too large.

Vanishing Gradient



Exploding Gradient

